

Geometry of IEEE-754 Binary Representation on $\mathbb{R}$

Kalaap Sadhu

Abstract

The digital computer operates within a discrete world of zeros and ones. However, continuous scientific computing requires dealing with real numbers spanning astronomical scales and subatomic domains. Because the real continuum $\mathbb{R}$ is uncountably infinite, representing it within finite binary strings is not possible. We have to choose some of them and this forces a fundamental philosophical and practical question: *How do we decide on the importance of a $\mathbb{R}$ number over the other?* This expository article traces the progression from unary counting and combinatorics of binary placeholders to floating-point schemes. We explore the geometry behind the IEEE-754 standard, illustrating why dynamic dyadic intervals $[2^E, 2^{E+1})$ provides the ideal set up of precision near zero while naturally discarding negligible fractional precision for large magnitudes.

Contents

1	Introduction: From Shepherds to Symbols	1
2	Combinatorics of Positional Tuples	2
2.1	Base-10 Tuples	2
2.2	The Binary System	3
2.3	Assigning Non-Negative Integers ($\mathbb{N}_0$)	3
3	The Real Continuum	4
3.1	The Uniform Partition Fallacy	4
3.2	The dilations of Dyadic Grids	5
4	The Architecture of IEEE-754	6
4.1	Assigning Mantissa Strings	6
4.2	Encoding the Exponent E with a Bias	6
4.3	Layout and Bitfields	7
4.4	Decoded Examples	7
5	Geometry of the Mantissa	8

1 Introduction: From Shepherds to Symbols

The computer speaks strictly in 0s and 1s. To facilitate any kind of mathematical operation, the very first step is to establish how to communicate the mathematical objects we are dealing with. Imagine teaching someone how to add when they do not even possess a concept of what numbers are.

What do the symbols 1, 2, and 3 even mean? Fundamentally, they are abstracted symbols representing collections containing that many physical objects. If this definition feels inherently cyclic, that is because it is.

To start from first principles, imagine an ancient shepherd who needs to ensure he departs for grazing and returns with all his sheep intact before counting was ever discovered. The shepherd devises a brilliant matching strategy using stones:

- For every sheep that exits the cave, he places a stone inside the cave.
- When he returns, he removes a stone for each sheep that re-enters the cave.
- If all stored stones are removed, all sheep have successfully returned.

This was inventing the **unary number system**. Math-folks would call this establishing a bijection relation between the set of sheep and set of stones. Abstracting the stones into a symbolic token ‘0’, a flock of 50 sheep can be written simply as an aggregated chain of tokens:

$$\underbrace{00000 \dots 00000}_{50\text{-many}}$$

Thus, the unary system formalizes non-negative quantities through repetition:

$$\{0, 00, 000, 0000, 00000, \dots\}$$

2 Combinatorics of Positional Tuples

Instead of confining ourselves to one variable, we can introduce two variables, three variables, or ten variables. We humans are particularly fond of the 10-variable set up since that is how we understand numbers. It is called the *Decimal system*; for our purposes it is better to call them *base-10* representation.

2.1 Base-10 Tuples

We can choose any ten symbols similar to how in unary system we need not use 0, even stones can work. But, let us choose the ten familiar symbols $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$:

- **One-tuples:** $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- **Two-tuples:** Formed by pairing each symbol systematically ($10^2 = 100$ items):

00	10	20	...	90
01	11	21	...	91
02	12	22	...	92
03	13	23	...	93
04	14	24	...	94
05	15	25	...	95
⋮	⋮	⋮		⋮
09	19	29	...	99

- **Three-tuples:** $\{0 \text{ before all two-tuples}, 1 \text{ before all two-tuples}, \dots, 9 \text{ before all two-tuples}\}$:

0 00	0 10	...	0 90	1 00	1 10	...	1 90	...	9 00	9 10	...	9 90
0 01	0 11	...	0 91	1 01	1 11	...	1 91	...	9 01	9 11	...	9 91
0 02	0 12	...	0 92	1 02	1 12	...	1 92	...	9 02	9 12	...	9 92
0 03	0 13	...	0 93	1 03	1 13	...	1 93	...	9 03	9 13	...	9 93
0 04	0 14	...	0 94	1 04	1 14	...	1 94	...	9 04	9 14	...	9 94
0 05	0 15	...	0 95	1 05	1 15	...	1 95	...	9 05	9 15	...	9 95
0 06	0 16	...	0 96	1 06	1 16	...	1 96	...	9 06	9 16	...	9 96
0 07	0 17	...	0 97	1 07	1 17	...	1 97	...	9 07	9 17	...	9 97
0 08	0 18	...	0 98	1 08	1 18	...	1 98	...	9 08	9 18	...	9 98
0 09	0 19	...	0 99	1 09	1 19	...	1 99	...	9 09	9 19	...	9 99

We clearly see the 0 block, 1 block, etc.

2.2 The Binary System

Our interest is in the two-variable case: $\{0, 1\}$.

- **One-tuples:** $\{0, 1\}$
- **Two-tuples:** The systematic pairing will be 0 and 1 before the one-tuples.

0 0 1 0
0 1 1 1

- **Three-tuples:** Here the pairing will be, in the same essence as before, 0 and 1 before the two-tuples. We will get $2^3 = 8$ combinations.

0 00 1 00
0 01 1 01
0 10 1 10
0 11 1 11

With an n -tuple, we have 2^n unique possible strings built out of combinations of 0 and 1. They act as our “placeholders” for however we want to use them or whatever information we want to store.

2.3 Assigning Non-Negative Integers ($\mathbb{N}_0$)

Assigning meaning to strings of length 4 allows identifying 16 non-negative integers ($\mathbb{N}_0 = \{0, 1, 2, 3, 4, \dots\}$):

String	Assigned Value	String	Assigned Value
0000	0	1000	8
0001	1	1001	9
0010	2	1010	10
0011	3	1011	11
0100	4	1100	12
0101	5	1101	13
0110	6	1110	14
0111	7	1111	15

We can store $\{0, 1, \dots, 31\}$ with 5-strings (2^5), $\{0, 1, \dots, 63\}$ with 6-strings (2^6), and so on. The standard and universally recognized way to list the n-string 0 and 1 is where the last column alternates between 0 and 1 every step. The second-last column alternates between 0 and 1 every $2^1 = 2$ steps, the third column alternates between 0 and 1 every $2^2 = 4$ steps, and so on.

3 The Real Continuum

Let us assign $\mathbb{R}$ -numbers now with the scheme exactly as above. The first step is to list $\mathbb{R}$ -numbers so that we can assign unique strings as above, but we are stuck: $\mathbb{R}$ -numbers form such a massive set that we cannot list them. This is a standard Mathematic result from Analysis.

Cantor's Diagonal Argument : The set of real numbers in $(0, 1)$ is uncountable.

Idea : Suppose, for contradiction, that the real numbers in $(0, 1)$ can be exhaustively enumerated into a list $r_1, r_2, r_3, \dots$:

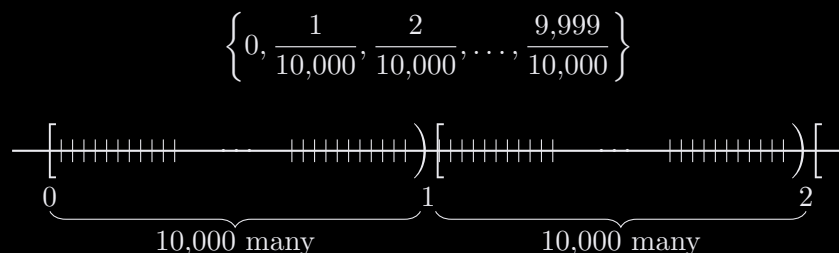
$$\begin{aligned} r_1 &= 0.d_{11}d_{12}d_{13}d_{14}d_{15} \dots \\ r_2 &= 0.d_{21}d_{22}d_{23}d_{24}d_{25} \dots \\ r_3 &= 0.d_{31}d_{32}d_{33}d_{34}d_{35} \dots \\ r_4 &= 0.d_{41}d_{42}d_{43}d_{44}d_{45} \dots \\ r_5 &= 0.d_{51}d_{52}d_{53}d_{54}d_{55} \dots \\ &\vdots \end{aligned}$$

$$\begin{array}{ll} r_1 = 0. \textcolor{blue}{5} & 1 \quad 0 \quad 7 \quad 2 \quad \dots \\ r_2 = 0. & 3 \quad \textcolor{blue}{8} \quad 2 \quad 0 \quad 9 \quad \dots \\ r_3 = 0. & 7 \quad 1 \quad \textcolor{blue}{4} \quad 8 \quad 3 \quad \dots \\ r_4 = 0. & 0 \quad 4 \quad 6 \quad \textcolor{blue}{1} \quad 5 \quad \dots \\ r_5 = 0. & 9 \quad 0 \quad 3 \quad 6 \quad \textcolor{blue}{6} \quad \dots \end{array} \quad \begin{array}{l} \text{Diagonal digits: } \{d_{nn}\} = \{5, 8, 4, 1, 6, \dots\} \\ \text{Define } x_n \neq d_{nn} \text{ (e.g., 6 if } d_{nn} = 5, \text{ else 5)} \\ \text{Constructed } x = 0.\textcolor{teal}{6}\textcolor{teal}{9}\textcolor{teal}{5}\textcolor{teal}{2}\textcolor{teal}{7} \dots \notin \text{List} \end{array}$$

Construct a new decimal $x = 0.x_1x_2x_3 \dots \in (0, 1)$ by setting $x_n = 6$ if $d_{nn} = 5$, and $x_n = 5$ otherwise. Then x differs from every r_n on the list at the n -th digit ($x \neq r_n$ for all n). Thus, x is not in the list, contradicting the assumption that $(0, 1)$ can be listed. Hence, $\mathbb{R}$ is uncountable.

3.1 The Uniform Partition Fallacy

One can solve this issue easily; we need not assign all $\mathbb{R}$ -numbers a string—only some of them. For instance, we could pick 10,000 points per interval $[n, n + 1)$ for $n \in \mathbb{N}_0$. In $[0, 1)$, we have 10,000 equally spaced points:



Similarly for $[1, 2), [2, 3), \dots$

However, keep in mind we use computers for huge calculations (such as rocket science, astrophysics, and optimization in thousands of variables). We must reach very large magnitudes, say up to $[0, 10^{40})$. That would require $10^4 \times 10^{40} = 10^{44}$ numbers.

Similarly, computers are used for atomic-scale calculations, so $\frac{1}{10,000}$ is nowhere near small enough. If we allow resolutions on the order of 10^{-10} , storing uniform intervals across large spans requires $10^{10} \times 10^{40} = 10^{50}$ states.

Not only does this consume huge storage, but it also wastes it. Imagine measuring the distance from our Sun to the closest star and caring whether the distance is:

$$10^7 \quad \text{versus} \quad 10^7 + \frac{1}{10^5}$$

At calculations of this magnitude, we do not care about differences of order 10^{-5} . We are wasting bits by assigning them distinct strings. Hence, we require three design criteria:

1. Record very large numbers.
2. Record very small numbers.
3. As we get closer to larger numbers, small differences are not stored; as we get toward 0, increasingly smaller differences are stored.

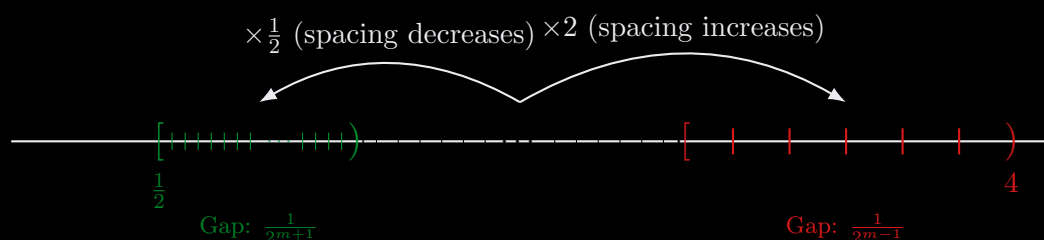
3.2 The dilations of Dyadic Grids

Keeping all this in mind, we create a fixed number of partitions in $[1, 2)$, and then scale it up as we go higher and scale it down as we go lower.

Suppose we consider 2^m partitions in $[1, 2)$. The numbers that will be stored are:

$$\left\{ 1, 1 + \frac{1}{2^m}, 1 + \frac{2}{2^m}, 1 + \frac{3}{2^m}, \dots, 2 - \frac{1}{2^m} \right\} \quad (2^m\text{-many})$$

We take this exact skeleton, multiply it by 2, and shift it right. Similarly, we multiply it by $\frac{1}{2}$ and shift it left:



Continuing this process:

- The interval $[2, 4)$ is multiplied by 2 and placed onto $[4, 8)$.
- The interval $[\frac{1}{2}, 1)$ is multiplied by $\frac{1}{2}$ and placed onto $[\frac{1}{4}, \frac{1}{2})$.

Every interval $[2^E, 2^{E+1})$ contains exactly 2^m points! This is what results is the number getting rarer as we increase the magnitude but gets closer as we go closer to 0.

4 The Architecture of IEEE-754

Now our number can be easily identified based on what power 2^E we are multiplying by and what the index within the 2^m partition is.

4.1 Assigning Mantissa Strings

Because the numbers in $[1, 2)$ are ordered, we can assign strings directly:

$$\begin{aligned}
 00 \dots 0000 &\longrightarrow 1 + (0 \times 2^{-m}) \\
 00 \dots 0001 &\longrightarrow 1 + (1 \times 2^{-m}) \\
 00 \dots 0010 &\longrightarrow 1 + (2 \times 2^{-m}) \\
 00 \dots 0011 &\longrightarrow 1 + (3 \times 2^{-m}) \\
 00 \dots 0100 &\longrightarrow 1 + (4 \times 2^{-m}) \\
 &\vdots \\
 11 \dots 1111 &\longrightarrow 1 + ((2^m - 1) \times 2^{-m}) = 2 - 2^{-m}
 \end{aligned}$$

We require an m -length string to store the 2^m numbers in $[1, 2)$. These m -length strings are called the ‘**Mantissa**’.

4.2 Encoding the Exponent E with a Bias

Next, we allocate an 8-bit string to E , giving $2^8 = 256$ states. We divide them between negative and non-negative powers (recall we want to multiply by 2^{10} as well as 2^{-10}):

$$E \in \{-128, -127, \dots, -1, 0, 1, \dots, 126, 127\} \quad (\text{total 256 values})$$

- $E = 0$ means the interval $[1, 2)$.
- $E = 1$ means the interval immediately right of $[1, 2)$, i.e., $[2, 4)$.
- $E = 2$ means $2^2 \times [1, 2) = [4, 8)$ interval.
- $E = -1$ means the interval immediately left of $[1, 2)$, i.e., $[\frac{1}{2}, 1)$.
- $E = -2$ means $2^{-2} \times [1, 2) = [\frac{1}{4}, \frac{1}{2})$ interval.

To eliminate explicit negative signs in binary representations, we perform a variable change:

$$e = E + 128 \implies e \in \{0, 1, 2, \dots, 255\}$$

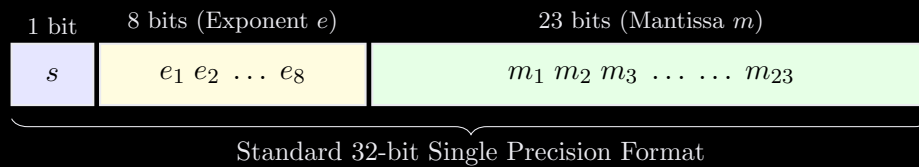
$$\begin{aligned}
 e = 0 &\implies E = -128 \quad (\text{leftmost interval: } [2^{-128}, 2^{-127})) \\
 e = 127 &\implies E = -1 \quad (\text{interval: } [2^{-1}, 1) = [\frac{1}{2}, 1)) \\
 e = 128 &\implies E = 0 \quad (\text{middle interval: } [1, 2)) \\
 e = 129 &\implies E = 1 \quad (\text{interval: } [2, 4)) \\
 e = 255 &\implies E = 127 \quad (\text{rightmost interval: } [2^{127}, 2^{128}))
 \end{aligned}$$

The 8-bit string e is called the ‘**Exponent**’.

Remark : Every time we move one dyadic interval to the right ($E \rightarrow E + 1$), the interval width doubles, but the number of representable grid points remains fixed at 2^{23} . As a direct consequence, the gap between neighboring representable numbers doubles at every single step. At 23^{rd} iteration, the gap size is exactly 1. So, 24^{th} iteration onward, we have the gap size at least 2 – *we start skipping $\mathbb{N}$ this step onward.*

4.3 Layout and Bitfields

At the very beginning, we place a sign bit (s): 0 denotes positive (+) and 1 denotes negative (-). This creates a mirrored structure to the left of zero.



The IEEE-754 specification standardizes bit allotments across several precisions:

Precision Name	Total Bits	Sign (s)	Exponent (e)	Mantissa (m)
Half Precision	16 bits	1	5	10
Single Precision	32 bits	1	8	23
Double Precision	64 bits	1	11	52
Quadruple Precision	128 bits	1	15	112
Octuple Precision	256 bits	1	19	236

For example, in 128-bit quadruple precision:

- $m = 112 \implies [1, 2)$ is partitioned into 2^{112} pieces.
- $e = 15 \implies 2^{15} = 32,768$ translational intervals (16,384 left and 16,383 right). Numbers reach down to 2^{-16384} and up to 2^{16384} .

4.4 Decoded Examples

Example 1: Consider the 32-bit string:

0 0000 0000 000000000000000000000010

- Sign bit is 0 $\implies$ positive (+).
- Exponent is $e = 0 \implies E = -128$ (we start on the right of 0 and start at the left most interval and then we move nowhere: $[2^{-128}, 2^{-127})$).
- Mantissa is binary 10 = 2, which addresses the 3rd number in the partition: $1 + \frac{2}{2^{23}}$.

$$\text{Value} = +1 \times 2^{-128} \times \left(1 + \frac{2}{2^{23}}\right) = \frac{1}{2^{128}} + \frac{2}{2^{151}}$$

Example 2: Consider the 32-bit string:

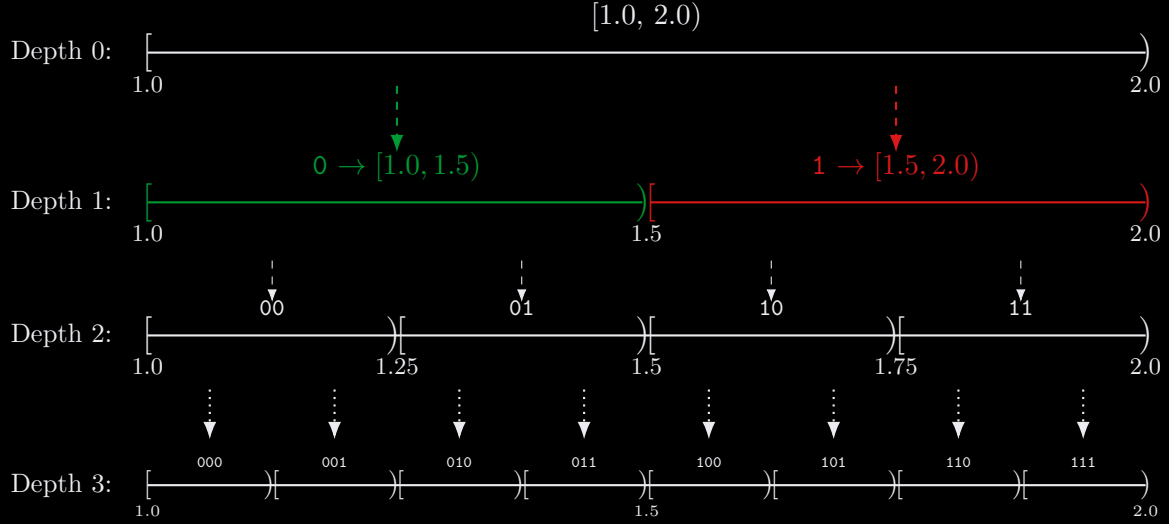
1 0000 0101 0000000000000000000000110

- Sign bit is 1 $\implies$ negative (-).
- Exponent is $e = 5 \implies E = 5 - 128 = -123$ (we start on the right of 0 and start at the left most interval, then 5th interval from there).
- Mantissa is binary 110 = 6, addressing the 7th number in the interval: $1 + \frac{6}{2^{23}}$.

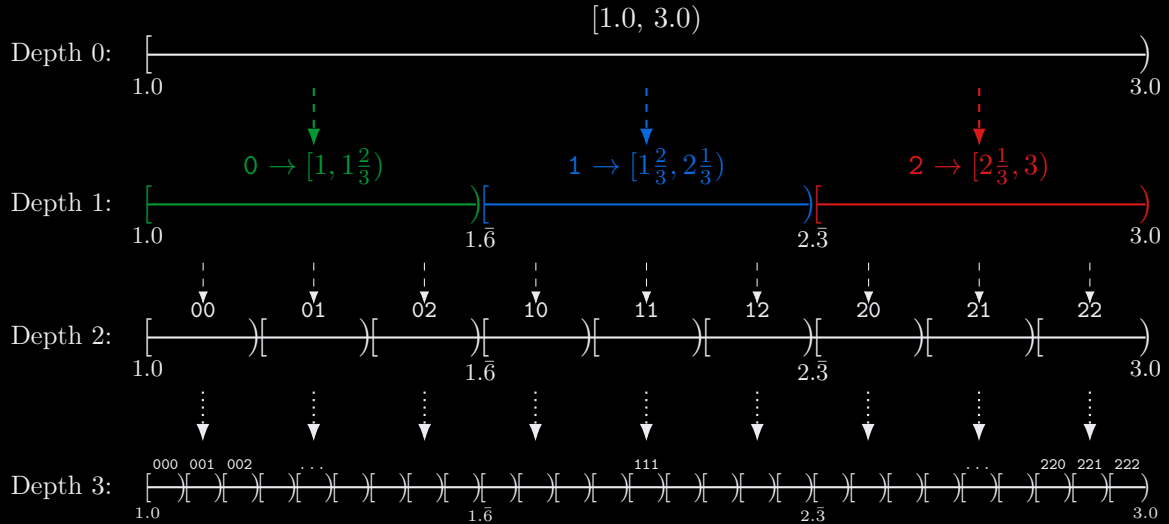
$$\text{Value} = -1 \times 2^{-123} \times \left(1 + \frac{6}{2^{23}}\right) = -\left(\frac{1}{2^{123}} + \frac{6}{2^{146}}\right)$$

5 Geometry of the Mantissa

We can focus exclusively on the canonical interval $[1, 2)$. We assume the interpretation that a bit value of 0 directs us to the left sub-interval, while 1 directs us to the right. A mantissa string of m bits is a sequence of m left/right choices, forming an exact spatial address on the dyadic subdivision of the real line.



A bitstring of m zeros and ones is a precise spatial address reached via successive binary zoom operations. Now that we have understood this idea, we will do what every mathematician loves to do – we extend this to other cases. Geometrically we can easily have this for **ternary system**. The variables will be $\{0, 1, 2\}$ where they will have the interpretation of left most, center and right most sub-intervals. A string of $\{0, 1, 2\}$ also gives us the a specific address based on which sub-interval to pick.



We have to move left and right by factor 3^E naturally here to get the entire storage system. Similarly, if we do this for $[1, 10]$ with 10 sub-divisions at every step we get exactly what Decimal system is and how we interpret numbers. We scale them by factor 10^E .

Lastly, in any of these systems what do you suppose happens when the partitions are allowed to be arbitrarily small? Firstly, it becomes theoretical and secondly, it gives us infinite precision.